

Database Concurrency - Explanation

What is Database Concurrency?

Concurrency in databases refers to the ability of a database system to allow multiple users or processes to access and modify data simultaneously without conflicts or data corruption. It's a fundamental aspect of multi-user database systems that ensures efficiency while maintaining data integrity.

Why is Concurrency Important?

In real-world applications, multiple users often need to interact with the same database at the same time. Without proper concurrency control:

- Data could be lost or corrupted
- Users might see inconsistent data
- The database could enter an invalid state
- Performance could degrade significantly

Key Concurrency Concepts

1. Transactions

A transaction is a sequence of database operations that are treated as a single unit of work. Transactions must follow ACID properties:

- **Atomicity:** All operations complete successfully or none do
- **Consistency:** Database moves from one valid state to another
- **Isolation:** Transactions don't interfere with each other
- **Durability:** Committed changes persist even after system failures

2. Isolation Levels

PostgreSQL supports four standard SQL isolation levels that control how transactions interact:

Read Uncommitted

- Lowest isolation level (not actually different from Read Committed in PostgreSQL)
- Transactions can see uncommitted changes from other transactions (dirty reads)

Read Committed (PostgreSQL default)

- Transactions only see data committed before the query began
- Prevents dirty reads
- Can still experience non-repeatable reads and phantom reads

Repeatable Read

- Ensures that if a transaction reads a row, subsequent reads will see the same data

- Prevents dirty reads and non-repeatable reads
- Can still experience phantom reads (new rows matching a condition)

Serializable

- Highest isolation level
- Transactions execute as if they were run one after another
- Prevents all concurrency anomalies
- May impact performance

3. Locks

Locks are mechanisms that prevent conflicts between concurrent transactions:

Lock Types:

- **Shared Locks (Read Locks):** Allow multiple transactions to read data simultaneously
- **Exclusive Locks (Write Locks):** Prevent other transactions from reading or writing while one transaction modifies data

Lock Granularity:

- **Row-level locks:** Lock individual rows
- **Table-level locks:** Lock entire tables
- **Page-level locks:** Lock data pages

4. Common Concurrency Problems

Lost Updates

When two transactions read the same data and then update it based on the value they read, one update can overwrite the other.

```
Transaction A: Read balance = $100
Transaction B: Read balance = $100
Transaction A: Update balance = $100 + $50 = $150
Transaction B: Update balance = $100 + $30 = $130
Result: Balance is $130 (lost the $50 addition!)
```

Dirty Reads

A transaction reads data that has been modified by another transaction but not yet committed.

```
Transaction A: Update balance to $150
Transaction B: Read balance = $150 (not committed!)
Transaction A: Rollback (balance returns to $100)
Transaction B: Uses $150 which was never valid
```

Non-Repeatable Reads

A transaction reads the same row twice but gets different values because another transaction modified and committed the data between the reads.

```
Transaction A: Read balance = $100
Transaction B: Update balance to $150 and COMMIT
Transaction A: Read balance = $150 (different value!)
```

Phantom Reads

A transaction executes a query twice and gets different sets of rows because another transaction added or removed rows.

```
Transaction A: Count accounts with balance > $100 = 5 rows
Transaction B: Insert new account with balance $200 and COMMIT
Transaction A: Count accounts with balance > $100 = 6 rows (phantom!)
```

PostgreSQL's Approach to Concurrency

PostgreSQL uses **Multi-Version Concurrency Control (MVCC)** which:

- Allows readers to never block writers and writers to never block readers
- Creates a new version of a row when it's updated (old version remains for concurrent transactions)
- Each transaction sees a consistent snapshot of the database
- Reduces the need for locks and improves performance

Best Practices for Concurrency

1. **Keep transactions short:** The longer a transaction runs, the more likely conflicts will occur
2. **Choose appropriate isolation levels:** Balance between consistency needs and performance
3. **Handle deadlocks gracefully:** Implement retry logic for deadlock situations
4. **Use explicit locking when necessary:** `SELECT ... FOR UPDATE` prevents others from modifying rows
5. **Monitor long-running queries:** They can block other transactions
6. **Design schema to minimize conflicts:** Partition data, use appropriate indexes

When to Use Different Isolation Levels

- **Read Committed:** Default for most applications, good balance of consistency and performance
- **Repeatable Read:** When you need consistent reads throughout a transaction (reports, analytics)
- **Serializable:** When absolute consistency is critical (financial transactions, inventory management)

Conclusion

Understanding concurrency is essential for building reliable, high-performance database applications. PostgreSQL's MVCC implementation provides excellent concurrency support, but developers must still understand isolation levels, locking mechanisms, and potential problems to design effective database interactions.